Microsoft

AI-200T00A

Deploy and monitor applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

Agenda

- Deploy applications to Azure Kubernetes Service
- Configure applications on Azure Kubernetes Service
- Monitor and troubleshoot applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

Module 1: Deploy applications to Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain how Deployments, Services, and Pods work together in AKS
- Create Kubernetes Deployment manifests to define how containerized applications run
- Write Service manifests to expose applications for internal or external access
- Use kubectl to deploy manifests and verify applications are running
- Troubleshoot common deployment issues using kubectl

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Managed Kubernetes control plane on Azure
- Runs containerized APIs, workers, and AI backends at scale
- Deployments maintain replica count and restart failed Pods
- Services provide stable connectivity despite Pod churn
- Azure load balancers expose public endpoints when needed
- kubectl is the primary tool for deploy, inspect, and repair operations

© Copyright Microsoft Corporation. All rights reserved.

Create Kubernetes deployment manifests

Desired state in YAML

- Define image, replica count, labels, and container port
- Keep Pod labels aligned with selector labels
- Treat the manifest as the source of truth for runtime intent

Runtime sizing

- Requests reserve schedulable CPU and memory
- Limits cap runaway usage and prevent noisy neighbors
- Replica count trades cost for availability and throughput

Configuration flow

- Use env vars for non-sensitive settings
- Reference Secrets for keys and credentials
- Avoid embedding secret values in the manifest

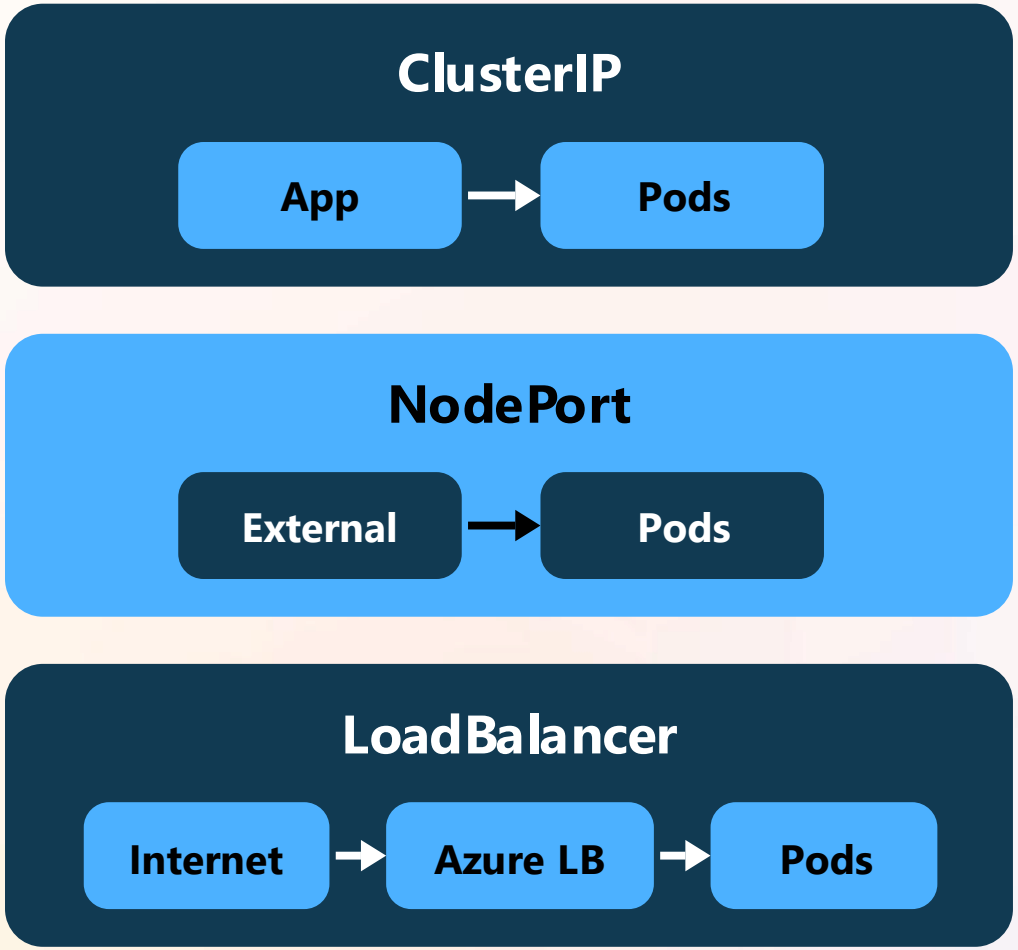
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-api    # must match
  template:
    metadata:
      labels:
        app: ai-api  # must match
    spec:
      containers:
      - name: ai-api
        image: myacr.azurecr.io/ai-api:v1
        # ports, resources, env, secrets
```

© Copyright Microsoft Corporation. All rights reserved.

Expose applications in Azure Kubernetes Services

- **ClusterIP:** Internal-only endpoint for app-to-app traffic
  - Best fit for back-end services inside the cluster
- **NodePort:** Opens a port on every node for basic external reachability
  - Useful for development and simple validation
- **LoadBalancer:** Provisions an Azure-managed public entry point
  - Best fit for production APIs serving external clients

- Service selector must match Pod labels
- `port` is the client-facing port
- `targetPort` is the container-facing port



© Copyright Microsoft Corporation. All rights reserved.

Deploy applications to Azure Kubernetes Services

Deployment flow

- Apply manifests and let the cluster reconcile state
- Wait for Pods to reach Running and replicas to become Ready
- Confirm the Service has endpoints and an address

Verification order

- Start with Pod and Deployment status
- Check Service exposure and endpoint assignment
- Read logs before changing configuration

Common failure signals

- ImagePullBackOff: Registry path or pull access issue
- CrashLoopBackOff: App starts, then exits or fails health checks
- Pending: Resource requests exceed available node capacity
- No endpoints: Service selector does not match Pod labels

© Copyright Microsoft Corporation. All rights reserved.

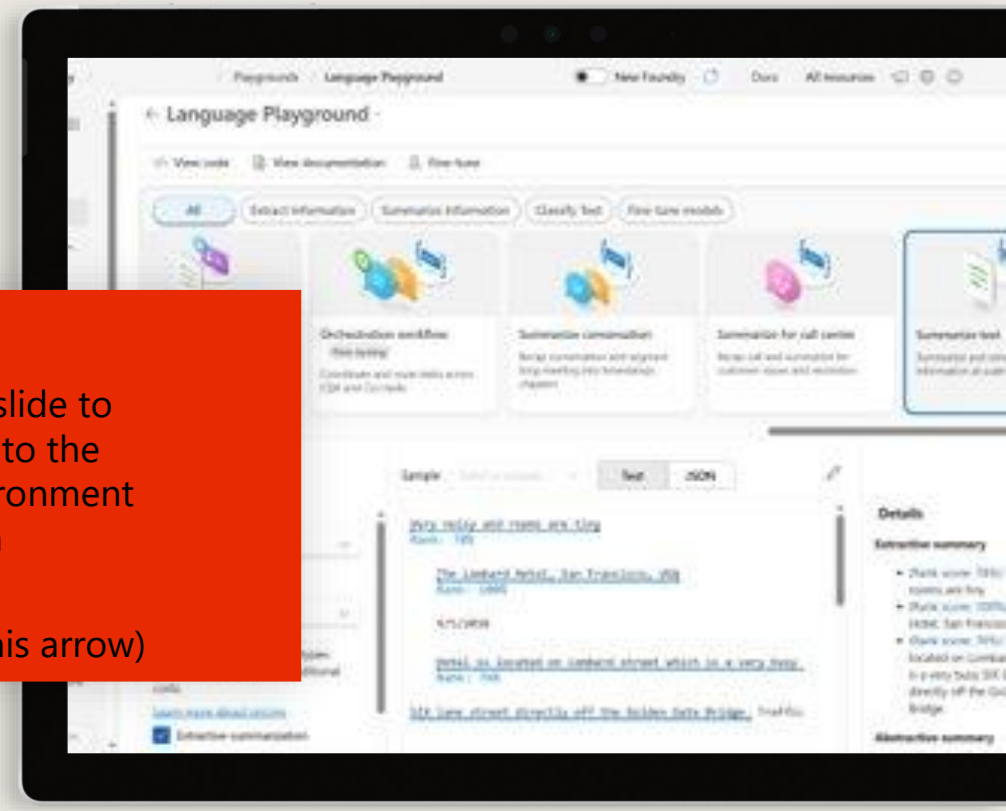
Exercise

Deploy an AI inference API to Azure Kubernetes Service

In this exercise, you deploy Azure resources including a Microsoft Foundry AI model, Azure Container Registry (ACR), and Azure Kubernetes Service (AKS) cluster.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2345722



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 1 knowledge check

1

Which Service type should you use when you need to expose an application to the internet with an Azure-managed load balancer?

A. LoadBalancer
B. ClusterIP
C. NodePort

2

What must match between a Deployment and a Service for traffic to route correctly?

A. The Pod labels in the Deployment template must match the Service selector
B. The container port in the Deployment must match the Service port
C. The Deployment name must match the Service name

© Copyright Microsoft Corporation. All rights reserved.

Module 1 summary

- **Deployment manifest:** Defines the desired runtime state for image, replicas, resources, and configuration
- **Requests and limits:** Drive scheduling behavior, stability, and total cluster cost
- **Service type:** Choose exposure scope first, then map ports correctly
- **Label matching:** Most important link between a Service and the Pods it should reach
- **Verification loop**: Check status, exposure, then logs before editing manifests
- **Troubleshooting signals:** Read Pod state names as clues to registry, startup, scheduling, or selector problems

© Copyright Microsoft Corporation. All rights reserved.

Module 2: Configure applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain how to externalize app configuration using Kubernetes primitives
- Implement ConfigMaps and inject settings into Pods
- Implement Secrets and consume sensitive values securely
- Attach persistent storage using PersistentVolume and PersistentVolumeClaim

© Copyright Microsoft Corporation. All rights reserved.

Introduction

- Externalize settings so containers stay portable across environments
- ConfigMaps store feature flags, endpoints, and nonsensitive values
- Secrets protect API keys, connection strings, and credentials
- PVCs provide durable storage that survives Pod restarts
- Azure integrations add Key Vault, App Configuration, and managed disks
- kubectl verifies that Pods receive the correct configuration

© Copyright Microsoft Corporation. All rights reserved.

Define ConfigMaps for application settings

Key-value configuration

- Nonsensitive settings stored as string pairs, up to 1 MiB
- Update without rebuilding containers

Consumption patterns

- Environment variables via configMapKeyRef
- Volume mounts for file-based config, auto-update on sync

Immutable option

- Prevents changes, reduces API server load
- Requires delete and recreate to modify

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-settings
data:
  FEATURE_X_ENABLED: "true"
  SERVICE_ENDPOINT:
"https://api.example.com"
  app.config: |
    log_level=info
    timeout_seconds=30
```

© Copyright Microsoft Corporation. All rights reserved.

Implement secrets for sensitive data

Kubernetes Secrets

- Store credentials, API keys, and connection strings
- Inject via secretKeyRef or mount as files

Azure Key Vault integration

- CSI driver mounts secrets directly from Key Vault into Pods
- Automated rotation and audit logging

Best practices

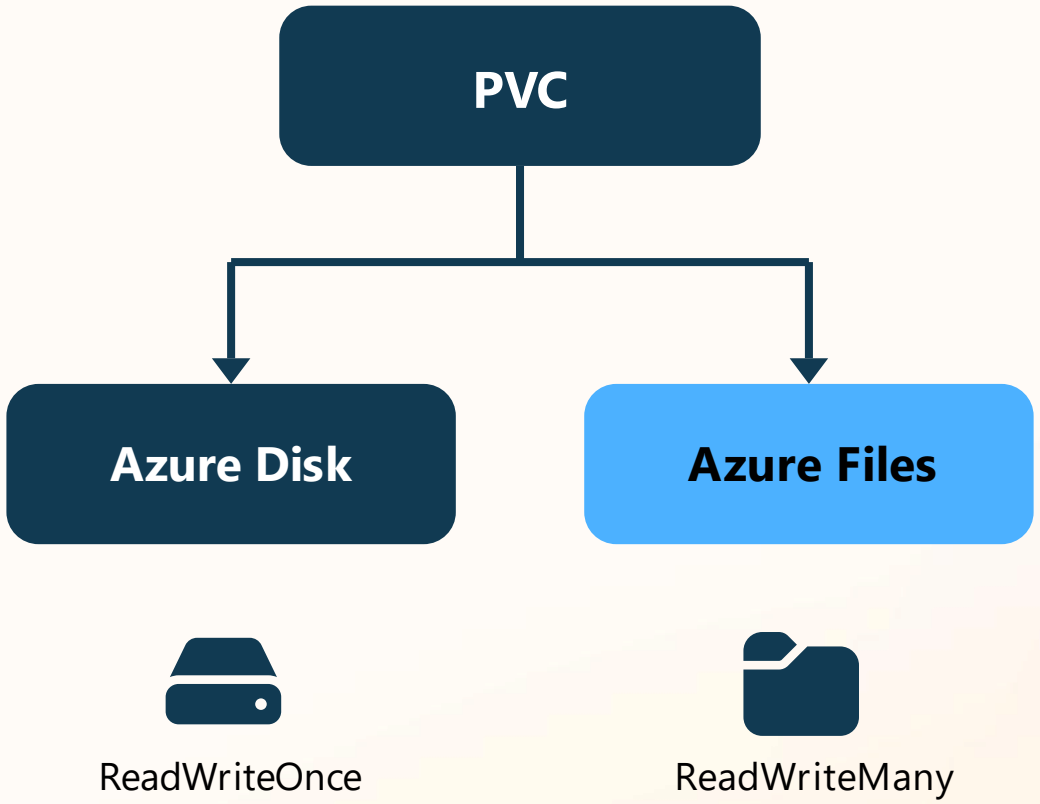
- Restrict access with RBAC, never commit literal values
- Rotate credentials regularly and trigger rollouts





© Copyright Microsoft Corporation. All rights reserved.

Attach persistent storage to an app



Storage model

- PVC requests storage with capacity and access mode
- StorageClass dynamically provisions Azure Disk or Azure Files

Access modes

- ReadWriteOnce: single-node block storage (Azure Disk)
- ReadWriteMany: shared file access across Pods (Azure Files)

Verification

- Confirm PVC status is Bound after applying
- Write a test file, delete the Pod, verify data persists

© Copyright Microsoft Corporation. All rights reserved.

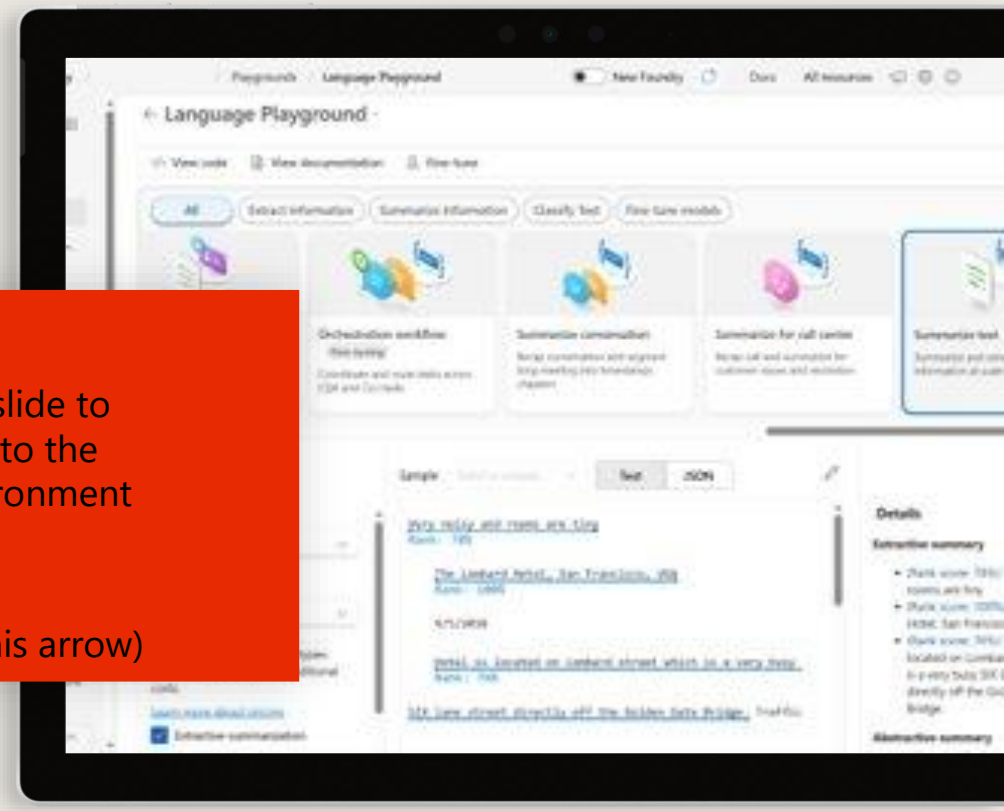
Exercise

Configure apps on Azure Kubernetes Service

In this exercise, you learn how to configure Kubernetes deployments with ConfigMaps for non-sensitive settings, Secrets for sensitive credentials, and PersistentVolumeClaims for persistent storage.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2346339



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 2 knowledge check

1

You need to store a database connection string that contains a password. The value shouldn't be visible in source control. Which Kubernetes resource should you use?

A. ConfigMap, because it stores configuration data
B. Secret, because it stores sensitive values and keeps credentials out of source control
C. PersistentVolumeClaim, because it provides storage for application data

2

Your application reads a JSON configuration file at startup. You want to update the file without rebuilding the container image. How should you provide the configuration?

A. Mount a ConfigMap as files using a volume so the JSON file appears on disk
B. Store the JSON in a Secret and inject it as an environment variable
C. Embed the JSON in the container image and rebuild when it changes

© Copyright Microsoft Corporation. All rights reserved.

Module 2 summary

- **ConfigMaps:** Externalize nonsensitive settings and update them without rebuilding images
- **Consumption patterns:** Choose environment variables for process-level settings, volume mounts for file-based configuration
- **Secrets:** Protect credentials with secretKeyRef injection and RBAC-restricted access
- **Key Vault integration:** Use the CSI driver for rotation, audit trails, and production-grade secret management
- **Persistent storage:** PVCs with StorageClasses provision Azure Disk or Azure Files automatically
- **Access mode decision:** ReadWriteOnce for single-Pod databases, ReadWriteMany for shared file access

© Copyright Microsoft Corporation. All rights reserved.

Module 3: Monitor and troubleshoot applications on Azure Kubernetes Service

© Copyright Microsoft Corporation. All rights reserved.

Learning objectives

- Explain key monitoring signals for AI workloads running on Azure Kubernetes Service
- Use kubectl and Azure tools to inspect logs and metrics for applications
- Troubleshoot pod and Service issues that affect AI APIs and workers
- Verify Service and ingress connectivity paths to ensure clients can reach AI services
- Apply a structured workflow to deploy, monitor, and debug applications on AKS

© Copyright Microsoft Corporation. All rights reserved.

Introduction

Observability on AKS

- Azure portal provides dashboards, Live Logs, and guided diagnostics
- kubectl gives direct access to logs, metrics, events, and pod details
- Identify latency spikes, error rates, and resource saturation early

Structured troubleshooting

- Inspect pod states and events to isolate the root cause
- Verify Service selectors, ports, and endpoint connectivity
- Fix manifests and redeploy with confidence

© Copyright Microsoft Corporation. All rights reserved.

Monitor application logs and metrics

Key signals

- Response latency, error rates, and pod restart counts
- CPU and memory versus configured requests and limits
- Container exit codes and health probe results

Observation tools

- Portal Monitoring tab and Container insights for cluster-wide views
- Live Logs to stream container output in real time
- kubectl logs and kubectl top for targeted CLI queries

Command examples

```
kubectl get pods -n ai-workloads
kubectl logs <pod-name> -n ai-workloads
kubectl logs -f <pod-name> -n ai-workloads
kubectl top pods -n ai-workloads
```

© Copyright Microsoft Corporation. All rights reserved.

Troubleshoot pods and services

Recognize failure states

- ImagePullBackOff: registry path or pull credentials
- CrashLoopBackOff: application exits or fails health checks
- Pending: resource requests exceed available node capacity

Investigation steps

- Describe the pod and scan the Events section for clues
- Check probes, environment variables, and mounted Secrets
- Use portal Diagnose and solve problems for guided fixes

1. Check pod status
2. Describe pod
3. Review logs
4. Debug container
5. Validate Service

© Copyright Microsoft Corporation. All rights reserved.

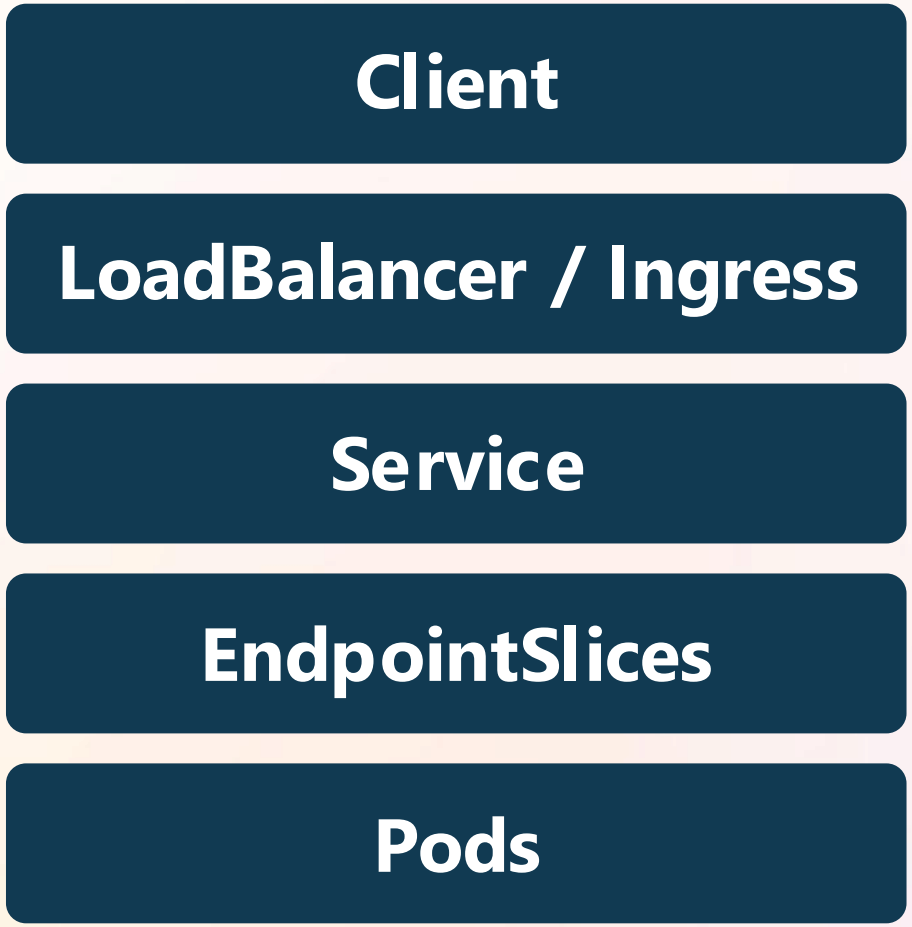
Verify service connectivity and endpoints

Service exposure

- ClusterIP for internal traffic, LoadBalancer for external access
- Selector labels must match pod labels to create endpoints
- Ports and targetPorts must align with container ports

Validation checks

- Inspect EndpointSlices to confirm pods back the Service
- Port-forward to test endpoints before external exposure
- Review Services and ingresses in the portal for external IPs



© Copyright Microsoft Corporation. All rights reserved.

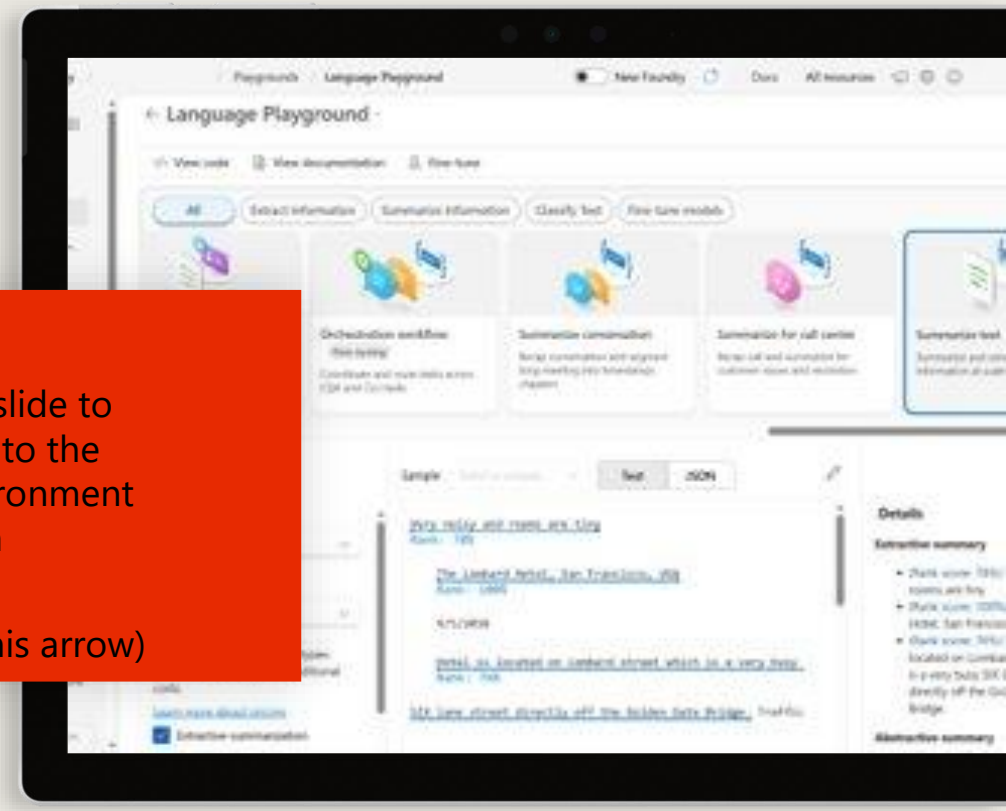
Exercise

Troubleshoot apps on Azure Kubernetes Service

In this exercise, you deploy a containerized API to Azure Kubernetes Service (AKS) and then diagnose and resolve common Kubernetes issues.

Estimated time: 30 minutes

Start the exercise at:
https://go.microsoft.com/fwlink/?linkid=2346293



Trainers:

Customize this slide to include the link to the hosted lab environment and any custom instructions.

(then remove this arrow)

© Copyright Microsoft Corporation. All rights reserved.

Module 3 knowledge check

1

An AI inference API returns intermittent HTTP 500 errors. You want to inspect error messages for a specific pod while reproducing the issue. Which approach is most appropriate?

A. Use kubectl logs -f to stream logs from the pod while sending test requests
B. Open Azure Monitor and review node CPU metrics for the last week
C. Run kubectl describe node on all nodes to check scheduling events

2

A Service shows no endpoints even though pods appear healthy. Which command confirms whether Service selectors match pod labels?

A. kubectl describe service
B. kubectl top nodes
C. kubectl logs

© Copyright Microsoft Corporation. All rights reserved.

Module 3 summary

- **Key signals first:** Monitor latency, error rates, restarts, and resource utilization before investigating deeper
- **Portal and kubectl together:** Use visual dashboards for overview, CLI for targeted investigation
- **Pod failure states:** ImagePullBackOff, CrashLoopBackOff, and Pending each point to a different root cause
- **Service connectivity chain:** Selector labels, port alignment, and EndpointSlices link Services to Pods
- **Verify before exposing:** Port-forward and internal testing confirm endpoints work before publishing

© Copyright Microsoft Corporation. All rights reserved.

Group discussion

© Copyright Microsoft Corporation. All rights reserved.

Group discussion questions

- Your AI inference API needs a model endpoint URL, an API key, and a 2 GB model file that persists across Pod restarts. Which Kubernetes resources handle each requirement, and what changes when you move to production?
- You deploy a new API version and pods enter CrashLoopBackOff with no Service endpoints. What do you check first, and how do you decide whether the problem is in the app, the configuration, or the cluster?
- Your public-facing API returns timeouts, but an internal ClusterIP service works fine inside the cluster. Where in the Client-to-Pod connectivity chain do you start investigating?

© Copyright Microsoft Corporation. All rights reserved.

End of presentation

© Copyright Microsoft Corporation. All rights reserved.